

4.29

1.ViT 介绍

Vision Transformer (ViT) 是由 Google 研究团队在 2020 年提出的一个里程碑式模型，它首次证明了纯粹的 Transformer 架构 (不使用卷积) 可以直接应用于图像分类，并在大规模数据上超越当时最先进的卷积神经网络 (CNN)。

2.具体结构

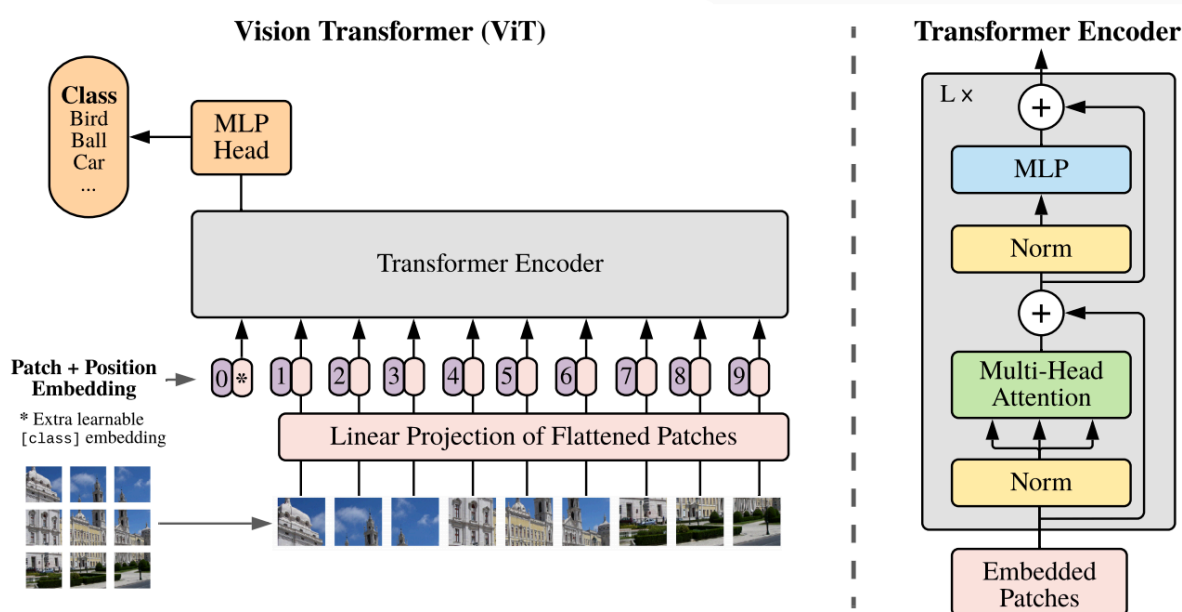


图2.1 ViT 的结构

Patch Embedding (图像分块与线性投影)

和 Transformer 相同，ViT 做了一次 Embedding。

NLP 里，Transformer 接收的是一维的单词序列。ViT 的做法就是把一张二维图像，切成一个个固定大小的图像块 (Patch)，每个 Patch 拉平成一个向量，作为 Transformer 的输入 Token。这样一来，图像分类就被转化成了一个标准的序列到标签的任务。

Position Embedding (位置编码)

- Transformer 对序列位置不敏感，需要显式告诉模型每个 Patch 原本在图像里的位置。
- 为每个 Patch 添加一个可学习的一维位置编码，直接加到 Patch Embedding 上。
- 实验表明，简单的 1D 可学习位置编码就能工作得很好，不需要复杂的 2D 编码。

与 Transformer 采用的位置编码不同，ViT 采用的位置编码是可学习的。

Class Token（分类标记）

- 借鉴 BERT 的 `[CLS]` 设计，ViT 在 Patch 序列前面拼接一个可学习的 `[class]` token。
- 这个特殊 token 本身不携带图像信息，但在经过多层 Transformer 后，其最终的输出向量会融合全局信息，直接送入分类头进行预测。

Transformer Encoder（编码器）

ViT 使用纯 Transformer 编码器（没有解码器），通过多层堆叠提取特征。每一层包含：

- 多头自注意力（Multi-Head Self-Attention, MSA）：让每个 Patch 都能和所有其他 Patch 交互，捕获全局依赖关系。
- 多层感知机（MLP）：通常由两个线性层和 GELU 激活函数组成。
- 每个子层前后都有 Layer Norm (LN) 和残差连接。

分类头（MLP Head）

- 取最后一层 `[class]` token 的输出向量，经过一个 Layer Norm 后，送入一个简单的线性分类器，输出类别概率。

3.具体代码

逐行 debug。

原本的代码训练速度非常慢，于是修改了一些超参数，把 ViT 的参数减少了，再增大 batch_size，提高显卡利用率，具体操作：

代码块

```
1  train_cifar10(  
2      img_size=32,  
3      patch_size=4,  
4      num_features=192,  
5      depth=4,  
6      num_heads=4,  
7      batch_size=512,  
8      num_workers=16,          # 根据 CPU 核心数调整  
9      epochs=20,  
10     lr=3e-4 * (512/32)      # 线性缩放学习率（可选，batch 大时可稍提高）  
11 )
```

让我们跟着代码来看看 ViT 的具体训练过程。

将二维图像展平

ViT首先将输入的二维图像分割成一系列固定大小的二维小块 (Patch) 。这个过程将图像从 $H \times W \times C$ (高度 $\times$ 宽度 $\times$ 通道数) 重塑为一个展平的二维图像块序列, 其形状为 $N \times (P^2 \cdot C)$ ¹ 。

- **图像分块 (Image Patching)**: 例如, ViT-Base模型通常将一张 $224 \times 224 \times 3$ 的图片, 划分为一个个 $16 \times 16 \times 3$ 的小块。这样图片在长宽维度上被划分为 14×14 的网格, 最终得到 $14 * 14 = 196$ 个图像块 ¹⁹。
- **块展平 (Patch Flattening)**: 接下来, 每个 $16 \times 16 \times 3$ 的三维图像块会被“拉平”成一个一维向量。这个向量的长度就是该块内所有像素点的数值总数, 即 $16 * 16 * 3 = 768$ ¹⁹。

经过上述操作后, 一张图片就变成了一个包含196个向量、每个向量长度为768的序列, 这个序列对于Transformer来说, 就如同文本句子中的词序列一样 ¹⁹。

图3.1 原论文采用的展平方法

这样展平之后还要映射到后续 Transformer 所采用的隐藏维度, 这一步线性映射是模型参数的一部分。

本采用了一种等效方法, 即卷积的方式, 我又修改了一下, 把这部分代码改为了原论文采用的线性映射的方法, 经过验证是可行的。

代码块

```
1 class VisionPatchEmbedding(nn.Module):
2     def __init__(self, image_size, patch_size, in_channels, embed_dim,
3         flatter=True):
4         super().__init__()
5         # 兼容image_size传入[C,H,W]或[H,W]的情况
6         if len(image_size) == 3:
7             self.image_size = image_size[1:] # (H, W)
8         else:
9             self.image_size = image_size
10        self.patch_size = patch_size
11        self.in_channels = in_channels
12        self.embed_dim = embed_dim
13        self.flatter = flatter
14
15        # 每个patch展平后的像素数, 即patch_size * patch_size * in_channels
16        self.patch_dim = (patch_size ** 2) * in_channels
17        # 原论文的核心: 用一个线性层将展平的patch投影到embed_dim
18        self.proj = nn.Linear(self.patch_dim, embed_dim)
19        self.norm = nn.LayerNorm(embed_dim)
20
21    def forward(self, x):
22        B, C, H, W = x.shape
23        pH, pW = self.image_size
24        assert H == pH and W == pW, f"Input size {H}x{W} doesn't match
25        expected {pH}x{pW}"
```

```

24         p = self.patch_size
25         h_patches = H // p
26         w_patches = W // p
27
28         # ---- 原论文的切分与展平 ----
29         # 将图像reshape为 (B, C, h_patches, p, w_patches, p)
30         x = x.reshape(B, C, h_patches, p, w_patches, p)
31         # 调整维度顺序, 将空间patch与通道+像素合并
32         x = x.permute(0, 2, 4, 1, 3, 5)          # (B, h_patches, w_patches,
C, p, p)
33         x = x.reshape(B, h_patches * w_patches, self.patch_dim) # (B,
num_patches, patch_dim)
34
35         # 线性投影到embed_dim
36         x = self.proj(x)                        # (B, num_patches, embed_dim)
37
38         if self.flatten:
39             # 此时已经为(B, N, embed_dim), 保持接口统一, 不需要额外操作
40             pass
41         x = self.norm(x)
42         return x

```

加入 Class Token

将 Class Token 扩展到整个 batch, 拼接到 patch token 最前面。

加上位置编码

输入 Transformer Block

这部分 ViT 也有创新点, 就是将归一化层前移。

```

class Block(nn.Module):
    """ViT的一个Transformer Block, 包含自注意力+MLP, 均采用Pre-Norm与残差连接"""
    def __init__(self, dim, num_heads, mlp_ratio, qkv_bias, drop, attn_drop, drop_path, act_layer, norm_layer):
        super().__init__()
        self.norm_1 = norm_layer(dim) # 自注意力前的LayerNorm
        self.attn = SelfAttention(dim, num_heads=num_heads, qkv_bias=qkv_bias,
                                   attn_drop_rate=attn_drop, proj_drop_rate=drop)
        self.norm_2 = norm_layer(dim) # MLP前的LayerNorm
        # hidden_features按比例mlp_ratio (通常为4) 扩大
        self.mlp = MLP(in_features=dim, hidden_features=int(dim * mlp_ratio),
                        out_features=None, act_layer=act_layer, drop_rate=drop_path)
        # 如果drop_path>0, 使用DropPath, 否则使用恒等映射
        self.drop_path = DropPath(drop_path) if drop_path > 0.0 else nn.Identity()

    def forward(self, x):
        # Pre-Norm自注意力, 残差连接 (经过drop_path)
        x = x + self.drop_path(self.attn(self.norm_1(x)))
        # Pre-Norm MLP, 残差连接
        x = x + self.drop_path(self.mlp(self.norm_2(x)))
        return x

```

图3.2 Transformer Block 的具体结构

再经过一层归一化层

完成之后，只取 class token 进行分类。



为什么可以只取 class token 进行分类？

在中间进行运算时，每个 class token 都与 patch token 进行过交互，可以理解为它已经关注过全局了，汇聚了整张图片的信息。

反向传播过程

优化器 Adam

维度	SGD (带动量)	Adam
学习率	全局统一，需手动细调	自适应，每个参数独立，默认设置通常就能工作
收敛速度	通常较慢，尤其在初期或梯度稀疏时	通常很快，能迅速越过平坦区
泛化能力	通常更优，更容易找到“平坦”的极小值，对未见过的数据更稳定	有时会收敛到“尖锐”的极小值，泛化性能稍逊
调参难度	非常敏感，学习率、动量、衰减率都需仔细调试	相对鲁棒，调参工作量较小
训练稳定性	后期震荡可能较大	训练过程通常更平滑、稳定
稀疏特征处理	困难，因为所有参数更新步调一致	出色，不常出现的特征也能得到有效更新

图3.3 SGD 与 Adam 比较

4.ViT 比 CNN 强在哪里

CNN 内置了平移不变性和局部性（卷积核只关注局部邻域），而 ViT 的自注意力层是全局的，对空间结构的先验假设很弱。

- 平移等变性：
指的是：输入移动到哪里，输出特征也会跟着移动到相同位置。
 - 核心：位置信息被保留了，只是同步移动了。
- 平移不变性：
指的是：无论输入怎么移动，最终输出的判断结果（或高层特征）都基本保持不变。
 - 核心：位置信息在高层被丢弃了。

CNN 如何实现平移不变性？

同一个一个卷积核在同一层上滑动时候参数不变，故忽略了位置信息。

ViT 不具有刚才提到的性质，是劣势也是优势

ViT 没有人为假定这些性质，也就是归纳偏置。

- 归纳偏置（Inductive Bias）：
它是模型在“看到数据之前”，就被赋予的一些先验假设或偏好。这些偏好会引导模型向特定的方向去学习，帮助它在有限的的数据中做出更合理的归纳。
- ViT 相当于通过注意力机制，把这些性质变成了一些可学习的参数，这些参数在通过大量学习之后产生的效果要比人为提前给定的这些性质要好，故而可以在精度上有所提升。

但是如果训练数据集不够大，模型没有来得及学会这些关系，就会使精度低于 CNN 等网络。

总结来说，上限高，下限低。